

Operating System Fundamentals

What is an Operating System?

An operating system (OS) is system software that manages computer hardware and software resources while providing common services to programs. It sits between the user and the hardware, acting as a resource manager and an extended machine. Without an OS, every application would need to communicate directly with hardware, making software development extremely difficult.

Core Responsibilities of an OS

Process management involves creating, scheduling, and terminating processes to ensure the CPU is utilized efficiently. Memory management allocates and reclaims RAM so multiple programs can run simultaneously without conflict. File system management organizes data on disks into files and directories. Device management provides a uniform interface to hardware peripherals through device drivers. Security and access control protect system resources from unauthorized use.

Types of Operating Systems

A Batch OS collects jobs and runs them without user interaction, ideal for repetitive tasks. A Time-Sharing OS allows multiple users to share the CPU by rapidly switching between processes, giving the illusion of simultaneous execution. A Real-Time OS guarantees responses within strict time deadlines, used in embedded systems like pacemakers and aircraft controls. A Distributed OS manages a group of independent computers so they appear as a single system. A Mobile OS, such as Android or iOS, is optimized for battery life and touch interfaces.

Kernel and User Space

The kernel is the core of the OS that runs with full hardware privileges. It handles the most critical tasks such as process scheduling and memory allocation. User space is where applications run with restricted privileges; they request kernel services through system calls. This separation protects the OS from faulty or malicious programs. A monolithic kernel includes all OS services in one large binary, while a microkernel keeps only the bare minimum in kernel space.

Process Management & Scheduling

Process vs Thread

A process is an independent program in execution. It has its own address space, registers, heap, and stack. Creating a process is expensive because the OS must allocate separate memory. A thread is a lightweight unit of execution within a process. Threads share the same memory space but have separate stacks and registers. Multi-threading allows a single application to perform multiple tasks concurrently, improving responsiveness and throughput.

Process States

Every process passes through several states during its lifetime. In the New state, the process is being created. In the Ready state, it is waiting for CPU time. In the Running state, the CPU is actively executing its instructions. In the Waiting state, it is blocked waiting for an I/O event or signal. In the Terminated state, execution is complete. The Process Control Block (PCB) stores all information about a process including its state, program counter, and memory limits.

CPU Scheduling Algorithms

First-Come First-Served (FCFS) processes jobs in arrival order. It is simple but can lead to long waiting times if a slow job arrives first (convoy effect). Shortest Job First (SJF) picks the process with the smallest burst time next, minimizing average waiting time but requiring knowledge of future burst times. Round Robin assigns a fixed time slice to each process in circular order, ensuring fairness. Priority Scheduling runs the highest-priority process first; lower-priority processes may starve. Multi-Level Feedback Queue dynamically adjusts a process's priority based on its behavior.

Deadlock

A deadlock is a situation where a set of processes are permanently blocked because each is waiting for a resource held by another. The four necessary Coffman conditions are mutual exclusion, hold and wait, no preemption, and circular wait. Deadlock prevention breaks one of these conditions by design. Deadlock avoidance uses algorithms like Banker's Algorithm to allocate resources safely. Deadlock detection allows deadlocks to occur but recovers by killing or rolling back processes.